Chapter 3. Decomposição de séries temporais

- Carregar bibliotecas e configurações comuns
- Carregar bibliotecas adicionais

Os dados de séries temporais podem exibir uma variedade de padrões, e muitas vezes é útil dividir uma série temporal em vários componentes, cada um representando uma categoria de padrão subjacente.

Na Seção 2.3, discutimos três tipos de padrões de séries temporais: tendência, sazonalidade e ciclos. Ao decompor uma série temporal em componentes, geralmente combinamos a tendência e o ciclo em um único componente **de tendência-ciclo** (frequentemente chamado apenas de **tendência** , por simplicidade). Assim, podemos pensar em uma série temporal como composta por três componentes: um componente de tendência-ciclo, um componente sazonal e um componente residual (que contém tudo o mais presente na série temporal). Para algumas séries temporais (por exemplo, aquelas observadas pelo menos diariamente), pode haver mais de um componente sazonal, correspondendo aos diferentes períodos sazonais.

Neste capítulo, consideramos os métodos mais comuns para extrair esses componentes de uma série temporal. Frequentemente, isso é feito para ajudar a melhorar a compreensão da série temporal, mas também pode ser usado para aprimorar a precisão da previsão.

Ao decompor uma série temporal, às vezes é útil primeiro transformar ou ajustar a série para tornar a decomposição (e a análise posterior) o mais simples possível. Portanto, começaremos discutindo transformações e ajustes.

3.1 Transformações e ajustes

Ajustar os dados históricos muitas vezes leva a uma série temporal mais simples. Aqui, abordamos quatro tipos de ajustes: ajustes de calendário, ajustes populacionais, ajustes de inflação e transformações matemáticas. O objetivo desses ajustes e transformações é simplificar os padrões nos dados históricos, removendo fontes conhecidas de variação ou tornando o padrão mais consistente em todo o conjunto de dados. Padrões mais simples geralmente são mais fáceis de modelar e levam a previsões mais precisas.

Ajustes de calendário

Parte da variação observada nos dados sazonais pode ser atribuída a efeitos simples do calendário. Nesses casos, geralmente é muito mais fácil remover a variação antes de realizar qualquer análise adicional.

Por exemplo, se você estiver analisando o total de vendas mensais de uma loja de varejo, haverá variação entre os meses simplesmente devido ao número diferente de dias úteis em cada mês, além da variação sazonal ao longo do ano. É fácil eliminar essa variação calculando a média de vendas por dia útil em cada mês, em vez do total de vendas no mês. Dessa forma, eliminamos efetivamente a variação sazonal.

Ajustes populacionais

Quaisquer dados afetados por mudanças populacionais podem ser ajustados para fornecer dados per capita. Ou seja, considere os dados por pessoa (ou por mil habitantes, ou por milhão de habitantes) em vez do total. Por exemplo, se você estiver estudando o número de leitos hospitalares em uma determinada região ao longo do tempo, os resultados serão muito mais fáceis de interpretar se você remover os efeitos das mudanças populacionais, considerando o número de leitos por mil habitantes. Assim, você poderá verificar se houve aumentos reais no número de leitos ou se os aumentos se devem inteiramente ao crescimento populacional. É possível que o número total de leitos aumente, mas o número de leitos por mil habitantes diminua. Isso ocorre quando a população está crescendo mais rapidamente do que o número de leitos hospitalares. Para a maioria dos dados afetados por mudanças populacionais, é melhor usar dados per capita em vez dos totais.

Isso pode ser observado no `global_economy` conjunto de dados, onde uma transformação comum do PIB é o PIB per capita.

```
global_economy = pd.read_csv("../data/global_economy.csv")

df = (
    global_economy
    .loc[lambda x: x["unique_id"] == "Australia"]
    .assign(y=lambda x: x["GDP"] / x["Population"])
)
plot_series(df, xlabel="Year [1Y]", ylabel="$US", title="GDP per capita")
```

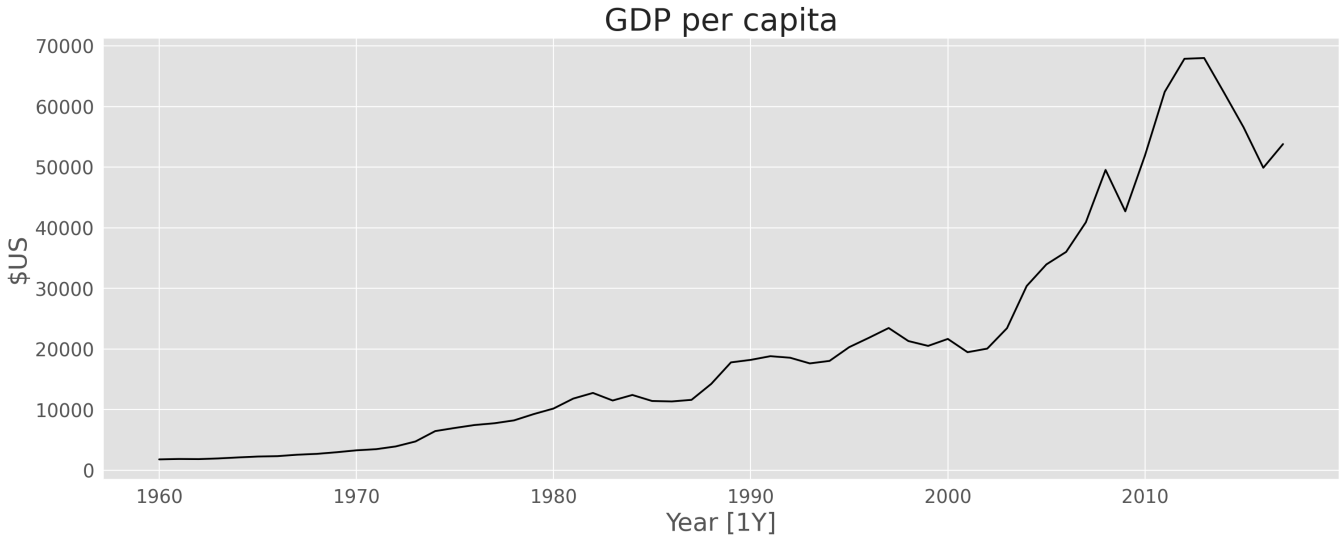


Figura 3.1: PIB per capita da Austrália.

Ajustes de inflação

Dados afetados pela variação cambial devem ser ajustados antes da modelagem. Por exemplo, o custo médio de uma casa nova aumentou nas últimas décadas devido à inflação. Uma casa de US$ 200.000 este ano não é a mesma que uma casa de US$ 200.000 vinte anos atrás. Por esse motivo, as séries temporais financeiras geralmente são ajustadas para que todos os valores sejam expressos em dólares de um determinado ano. Por exemplo, os dados de preços de imóveis podem ser expressos em dólares do ano 2000.

Para fazer esses ajustes, utiliza-se um índice de preços. Se$z_t$denota o índice de preços e$y_t$indica o preço original da casa no ano$t$, então$x_t = y_t / \ z_t * z_{2000}$ gives the adjusted house price at year 2000 dollar values. Price indexes are often constructed by government agencies. For consumer goods, a common price index is the Consumer Price Index (or CPI).

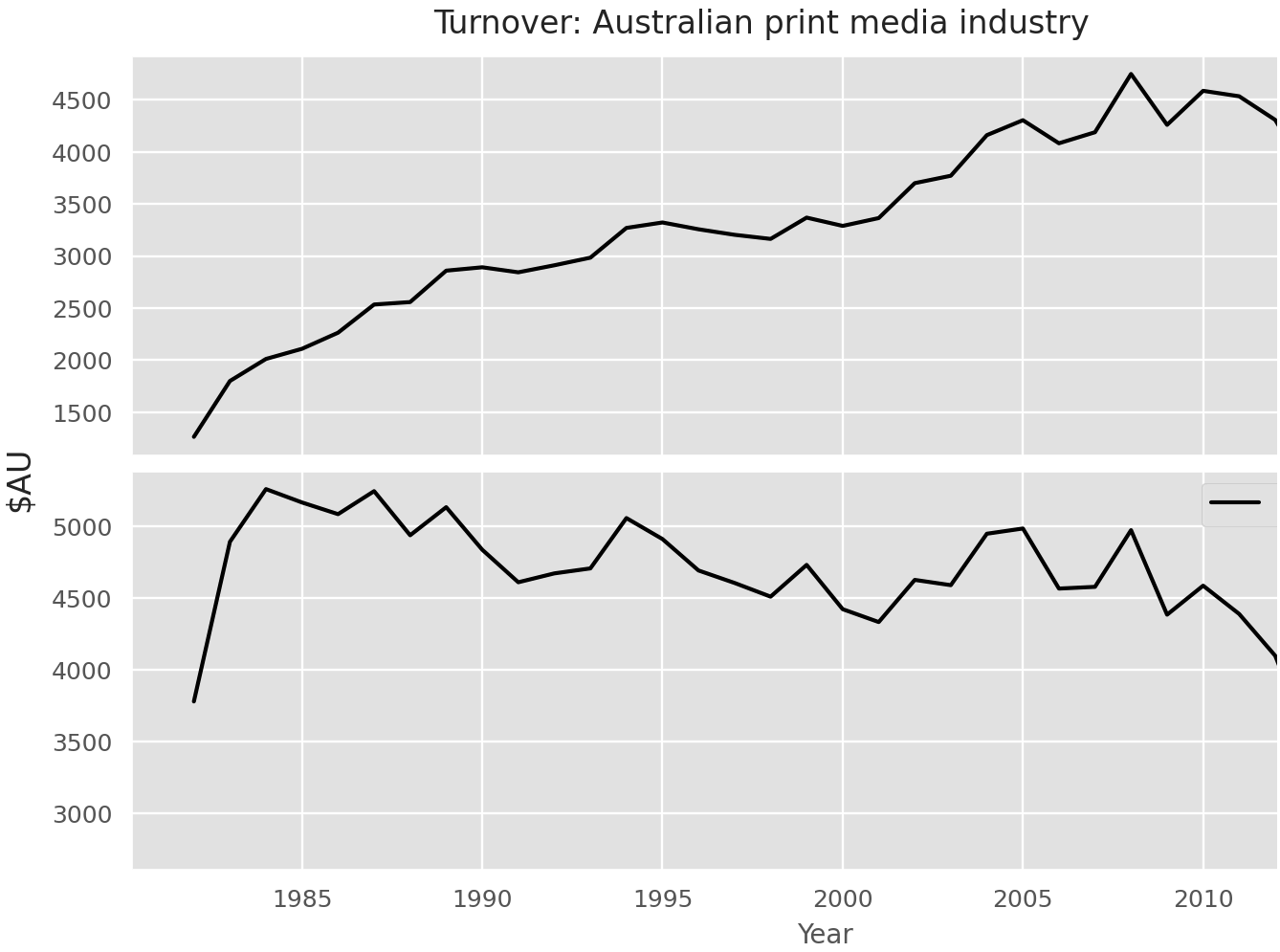
This allows us to compare the growth or decline of industries relative to a common price value. For example, looking at aggregate annual “newspaper and book” retail turnover from `aus_retail`, and adjusting the data for inflation using CPI from `global_economy` allows us to understand the changes over time.

```
aus_retail = pd.read_csv("../data/aus_retail.csv", parse_dates=["Month"])
print_retail = (
    aus_retail
    .loc[lambda x: x["Industry"] == "Newspaper and book retailing"]
    .assign(ds=lambda x: x["Month"].dt.year)
    .groupby("ds", as_index=False)
    ["Turnover"].sum()
)

aus_economy = global_economy.loc[lambda x: x["unique_id"] == "Australia"]
df = (
    aus_economy
    .merge(print_retail, on="ds", how="left")
```

```
    .assign(adjusted_turnover=lambda x: x["Turnover"] / x["CPI"] * 100)
    .dropna()
)

fig, axes = plt.subplots(2, 1, sharex=True)
sns.lineplot(data=df, x='ds', y='Turnover', ax=axes[0],
    color='black', label="Turnover")
sns.lineplot(data=df, x='ds', y='adjusted_turnover', ax=axes[1],
    color='black', label="Adjusted_turnover")
axes[0].set(ylabel="")
axes[1].set(ylabel="", xlabel='Year')
fig.suptitle("Turnover: Australian print media industry")
fig.supylabel("$AU")
plt.show()
```



By adjusting for inflation using the CPI, we can see that Australia's newspaper and book retailing industry has been in decline much longer than the original data suggests. The adjusted turnover is in 2010 Australian dollars, as CPI is 100 in 2010 in this data set.

Mathematical transformations

If the data shows variation that increases or decreases with the level of the series, then a transformation can be useful. For example, a logarithmic transformation is often useful. If we denote the original observations as $y_1, \ldots, y_T$ and the transformed observations as $w_1, \ldots, w_T$,

then $w_t = \log(y_t)$. Logarithms are useful because they are interpretable: changes in a log value are relative (or percentage) changes on the original scale. So if log base 10 is used, then an increase of 1 on the log scale corresponds to a multiplication of 10 on the original scale. If any value of the original series is zero or negative, then logarithms are not possible.

Sometimes other transformations are also used (although they are not so interpretable). For example, square roots and cube roots can be used. These are called **power transformations** because they can be written in the form $w_t = y_t^p$.

A useful family of transformations, that includes both logarithms and power transformations, is the family of **Box-Cox transformations** (Box and Cox 1964), which depend on the parameter λ and are defined as follows:

$$w_t = \begin{cases} \log(y_t) & \text{if } \lambda = 0; \\ (\text{sign}(y_t)|y_t|^\lambda - 1)/\lambda & \text{otherwise.} \end{cases} \tag{3.1}$$

This is actually a modified Box-Cox transformation, discussed in Bickel and Doksum (1981), which allows for negative values of y_t provided $\lambda > 0$.

The logarithm in a Box-Cox transformation is always a natural logarithm (i.e., to base e). So if $\lambda = 0$, natural logarithms are used, but if $\lambda \neq 0$, a power transformation is used, followed by some simple scaling.

If $\lambda = 1$, then $w_t = y_t - 1$, so the transformed data is shifted downwards but there is no change in the shape of the time series. For all other values of λ, the time series will change shape.

A good value of λ is one which makes the size of the seasonal variation about the same across the whole series, as that makes the forecasting model simpler.

To determine a λ for the Box-Cox transformation, use the `boxcox_lambda()` function from `CoreForecast`. This function requires the `season_length` of the series and uses the method by Guerrero (1993) to choose a value for λ.

You can apply the Box-Cox transformation directly to the data using the `boxcox()` function from `CoreForecast`. This function accepts any λ you provide, including the optimal value determined by `boxcox_lambda()`.

```
aus_production = pd.read_csv("../data/aus_production.csv",
    parse_dates=["ds"])
aus_gas = (
    aus_production[["ds", "Gas"]]
    .rename(columns={"Gas": "y"})
    .assign(unique_id="Gas")
)

y = aus_gas["y"].to_numpy()
optim_lambda = boxcox_lambda(y, method="guerrero", season_length=4)
```

```
df = aus_gas.assign(y_transformed=boxcox(y, optim_lambda))

title = rf"Transformed gas production with $\lambda$ = {optim_lambda:.2f}"
plot_series(df, target_col="y_transformed",
    xlabel="Quarter [1Q]", ylabel="", title=title)
```

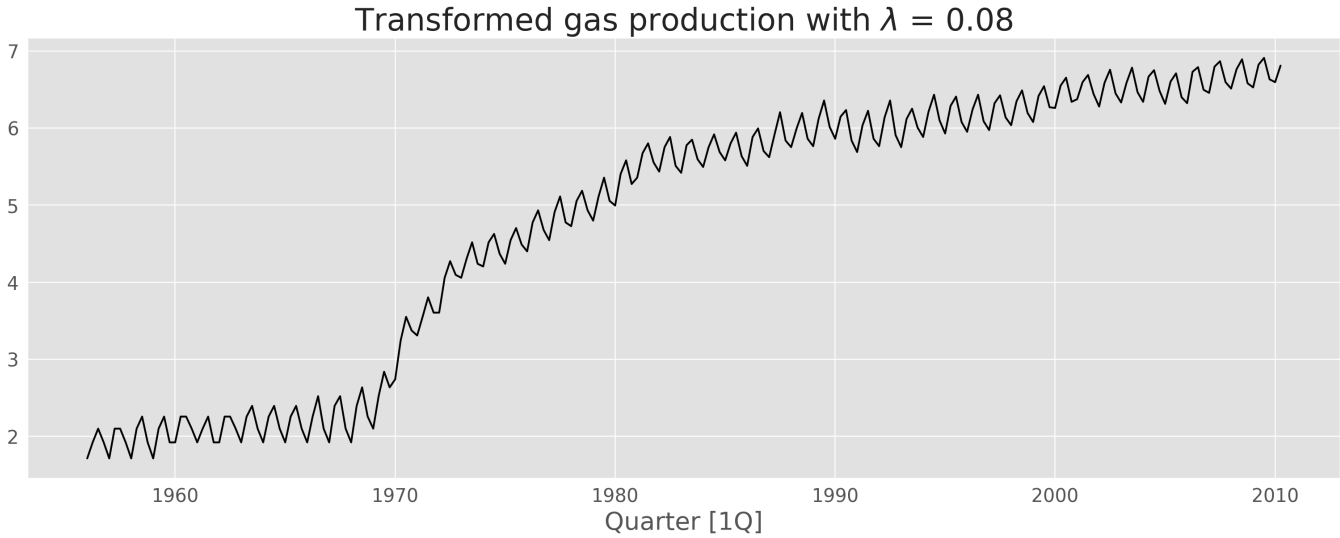


Figure 3.2: Transformed Australian gas production with λ chosen using the `boxcox_lambda()` function from `CoreForecast`.

3.2 Time series components

If we assume an additive decomposition, then we can write

$$y_t = S_t + T_t + R_t,$$

where y_t is the data, S_t is the seasonal component, T_t is the trend-cycle component, and R_t is the remainder component, all at period t. Alternatively, a multiplicative decomposition would be written as

$$y_t = S_t \times T_t \times R_t.$$

The additive decomposition is the most appropriate if the magnitude of the seasonal fluctuations, or the variation around the trend-cycle, does not vary with the level of the time series. When the variation in the seasonal pattern, or the variation around the trend-cycle, appears to be proportional to the level of the time series, then a multiplicative decomposition is more appropriate. Multiplicative decompositions are common with economic time series.

An alternative to using a multiplicative decomposition is to first transform the data until the variation in the series appears to be stable over time, then use an additive decomposition. When a log transformation has been used, this is equivalent to using a multiplicative decomposition

on the original data because

$$y_t = S_t \times T_t \times R_t \quad \text{is equivalent to} \quad \log y_t = \log S_t + \log T_t + \log R_t.$$

Example: Employment in the US retail sector

We will look at several methods for obtaining the components S_t, T_t and R_t later in this chapter, but first it is helpful to see an example. We will decompose the number of persons employed in retail as shown in Figure 3.3. The data shows the total monthly number of persons in thousands employed in the retail sector across the US since 1990.

```
us_employment = pd.read_csv("../data/us_employment.csv", parse_dates=["ds"])
us_retail_employment = us_employment.loc[lambda x:
    (x["unique_id"] == "Retail Trade") & (x["ds"] >= "1990")
]

plot_series(us_retail_employment,
    xlabel="Month [1M]", ylabel="Persons (thousands)",
    title="Total employment in US retail")
```

Figure 3.3: Total number of persons employed in US retail.

To illustrate the ideas, we will use the STL decomposition method, which is discussed in Section 3.6.

```
stl = STL(us_retail_employment["y"], period=12)
res = stl.fit()

dcmp = pd.DataFrame({
    "ds": us_retail_employment["ds"],
    "data": us_retail_employment["y"],
    "trend": res.trend,
    "seasonal": res.seasonal,
    "remainder": res.resid,
}).reset_index(drop=True)
```

```
dcmp.head()
```

	ds	data	trend	seasonal	remainder
0	1990-01-01	13256	13296	-4	-36.7
1	1990-02-01	12966	13276	-288	-21.4
2	1990-03-01	12938	13256	-307	-10.8
3	1990-04-01	13012	13235	-236	13.1
4	1990-05-01	13108	13214	-115	9.6

The output above shows the components of an STL decomposition. The original data is shown (as `data`), followed by the estimated components.

The `trend` column (containing the trend-cycle T_t) follows the overall movement of the series, ignoring any seasonality and random fluctuations, as shown in Figure 3.4.

```
fig, ax = plt.subplots()
sns.lineplot(data=dcmp, x="ds", y="data", color="gray", label="Data")
sns.lineplot(data=dcmp, x="ds", y="trend", color="#D55E00", label="Trend")
ax.set(
    title="Total employment in US retail",
    xlabel="Month [1M]",
    ylabel="Persons (thousands)",
)
plt.show()
```

Figure 3.4: Total number of persons employed in US retail: the trend-cycle component (orange) and the raw data (grey).

We can plot all of the components of the decomposition, as shown in Figure 3.5.

```
fig, axes = plt.subplots(4, 1, sharex=True, figsize=(8, 6))
sns.lineplot(data=dcmp, x="ds", y="data", ax=axes[0])
sns.lineplot(data=dcmp, x="ds", y="trend", ax=axes[1])
sns.lineplot(data=dcmp, x="ds", y="seasonal", ax=axes[2])
sns.lineplot(data=dcmp, x="ds", y="remainder", ax=axes[3])
axes[0].set_title("Employed = trend + seasonal + remainder",
    size="medium", loc="left")
axes[0].set(ylabel="Employed")
axes[3].set(xlabel="")
fig.suptitle("STL decomposition")
plt.show()
```

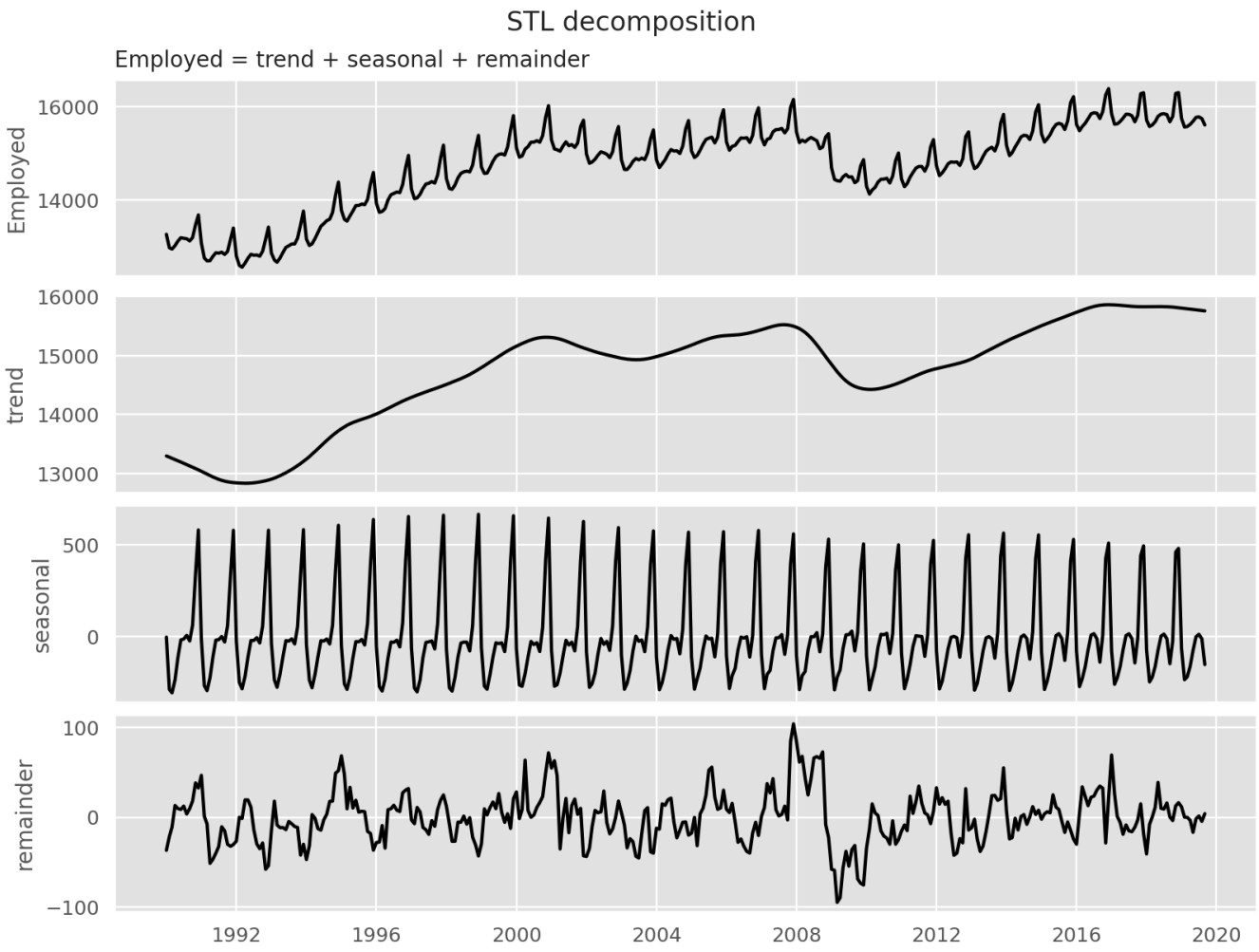


Figure 3.5: The total number of persons employed in US retail (top) and its three additive components.

The three components are shown separately in the bottom three panels. These components can be added together to reconstruct the data shown in the top panel. Notice that the seasonal component changes over time, so that any two consecutive years have similar patterns, but years far apart may have different seasonal patterns. The remainder component shown in the

bottom panel is what is left over when the seasonal and trend-cycle components have been subtracted from the data.

Seasonally adjusted data

If the seasonal component is removed from the original data, the resulting values are the "seasonally adjusted" data. For an additive decomposition, the seasonally adjusted data are given by $y_t - S_t$, and for multiplicative data, the seasonally adjusted values are obtained using y_t / S_t.

Figure 3.6 shows the seasonally adjusted number of persons employed.

```
df = dcmp.assign(adj=lambda x: x["data"] - x["seasonal"])

fig, ax = plt.subplots()
sns.lineplot(data=df, x="ds", y="data",
    color="gray", label="Data")
sns.lineplot(data=df, x="ds", y="adj",
    color="#0072B2", label="Seasonally adjusted data")
ax.set(
    title="Total employment in US retail",
    xlabel="Month [1M]",
    ylabel="Persons (thousands)",
)
plt.show()
```

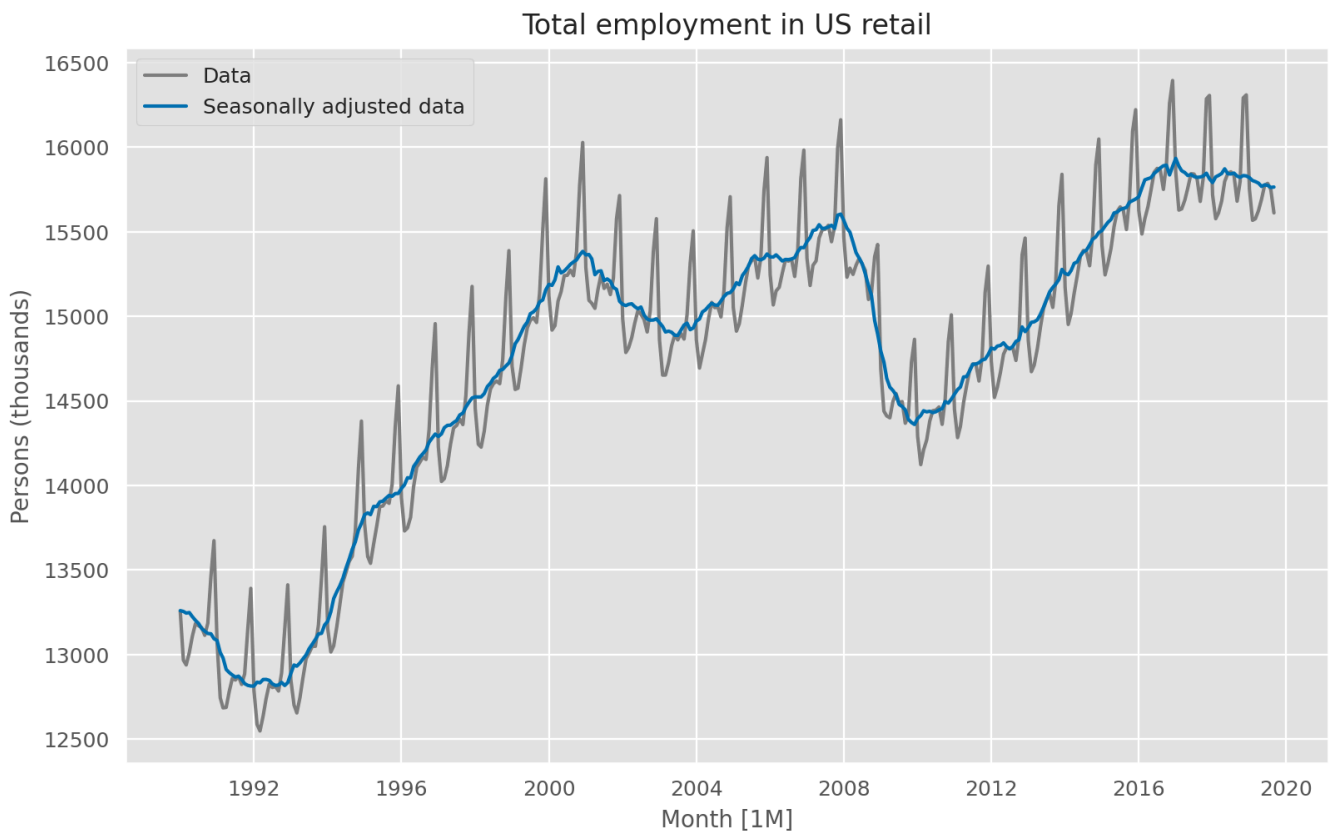


Figure 3.6: Seasonally adjusted retail employment data (blue) and the original data (grey).

If the variation due to seasonality is not of primary interest, the seasonally adjusted series can be useful. For example, monthly unemployment data are usually seasonally adjusted in order to highlight variation due to the underlying state of the economy rather than the seasonal variation. An increase in unemployment due to school leavers seeking work is seasonal variation, while an increase in unemployment due to an economic recession is non-seasonal. Most economic analysts who study unemployment data are more interested in the non-seasonal variation. Consequently, employment data (and many other economic series) are usually seasonally adjusted.

Seasonally adjusted series contain the remainder component as well as the trend-cycle. Therefore, they are not "smooth", and "downturns" or "upturns" can be misleading. If the purpose is to look for turning points in a series, and interpret any changes in direction, then it is better to use the trend-cycle component rather than the seasonally adjusted data.

3.3 Moving averages

The classical method of time series decomposition originated in the 1920s and was widely used until the 1950s. It still forms the basis of many time series decomposition methods, so it is important to understand how it works. The first step in a classical decomposition is to use a moving average method to estimate the trend-cycle, so we begin by discussing moving averages.

Moving average smoothing

A moving average of order m can be written as

$$\hat{T}_t = \frac{1}{m} \sum_{j=-k}^{k} y_{t+j}, \tag{3.2}$$

where $m = 2k + 1$. That is, the estimate of the trend-cycle at time t is obtained by averaging values of the time series within k periods of t. Observations that are nearby in time are also likely to be close in value. Therefore, the average eliminates some of the randomness in the data, leaving a smooth trend-cycle component. We call this an m-**MA**, meaning a moving average of order m.

For example, consider Figure 3.7 which shows exports of goods and services for Australia as a percentage of GDP from 1960 to 2017. The data are also shown in the table below.

```
global_economy = pd.read_csv("../data/global_economy.csv")
df = global_economy.loc[lambda x: x["unique_id"] == "Australia"]

plot_series(df, target_col="Exports",
```

```
    xlabel="Year [1Y]", ylabel="% of GDP",
    title="Total Australian exports")
```

Figure 3.7: Australian exports of goods and services: 1960–2017.

Year	Exports	5-MA
1960	12.99	
1961	12.40	
1962	13.94	13.46
1963	13.01	13.50
1964	14.94	13.61
1965	13.22	13.40
1966	12.93	13.25
1967	12.88	12.66
…	…	…
2010	19.84	21.21
2011	21.47	21.17
2012	21.52	20.78
2013	19.99	20.81
2014	21.08	20.37
2015	20.01	20.32
2016	19.25	
2017	21.27	

In the last column of this table, a moving average of order 5 is shown, providing an estimate of the trend-cycle. The first value in this column is the average of the first five observations, 1960–1964; the second value in the 5-MA column is the average of the values for 1961–1965; and so on. Each value in the 5-MA column is the average of the observations in the five year window centred on the corresponding year. In the notation of Equation 3.2, column 5-MA contains the values of $\hat{T}_t$ with $k = 2$ and $m = 2k + 1 = 5$. There are no values for either the first two years or the last two years, because we do not have two observations on either side. Later we will use more sophisticated methods of trend-cycle estimation which do allow estimates near the endpoints.

This is easily computed using the `rolling()` function from `pandas`. In this case, we use a window of size 5. Note that we need to set `center=True` so that the moving average is centered around each observation.

```
aus_exports = (
    global_economy
    .loc[lambda x: x["unique_id"] == "Australia"]
    .assign(MA_5=lambda x: x["Exports"].rolling(5, center=True).mean())
)
```

To see what the trend-cycle estimate looks like, we plot it along with the original data in Figure 3.8.

```
fig, ax = plt.subplots()
sns.lineplot(data=aus_exports, x="ds", y="Exports",
    color="black", label="Exports")
sns.lineplot(data=aus_exports, x="ds", y="MA_5",
    color="#D55E00", label="5-MA")
ax.set(
    title="Total Australian exports",
    xlabel="Year",
    ylabel="% of GDP",
)
plt.show()
```

Figure 3.8: Australian exports (black) along with the 5-MA estimate of the trend-cycle (orange).

Notice that the trend-cycle (in orange) is smoother than the original data and captures the main movement of the time series without all of the minor fluctuations. The order of the moving average determines the smoothness of the trend-cycle estimate. In general, a larger order means a smoother curve. Figure 3.9 shows the effect of changing the order of the moving average for the Australian exports data.

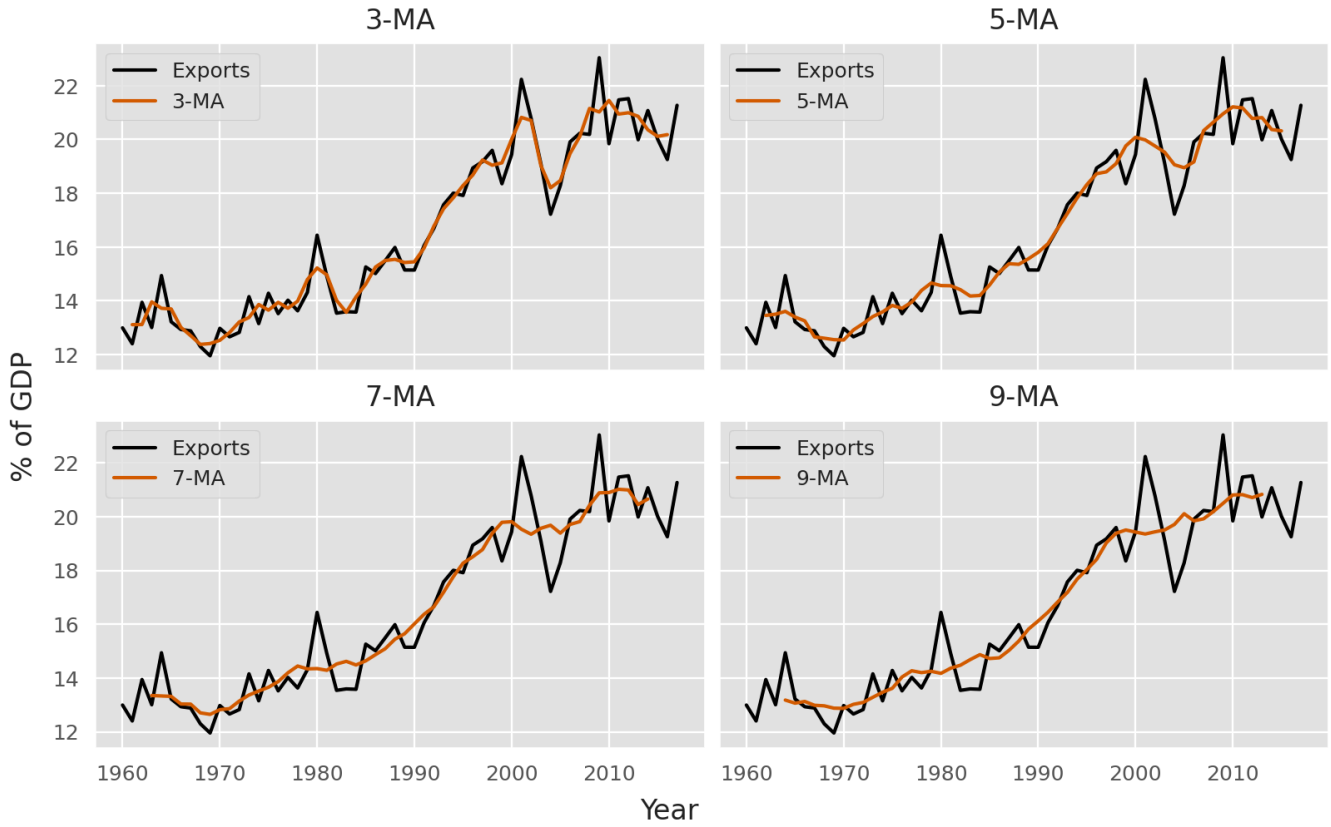


Figure 3.9: Different moving averages applied to the Australian exports data.

Simple moving averages such as these are usually of an odd order (e.g., 3, 5, 7, etc.). This is so they are symmetric: in a moving average of order $m = 2k + 1$, the middle observation, and k observations on either side, are averaged. But if m was even, it would no longer be symmetric.

Moving averages of moving averages

It is possible to apply a moving average to a moving average. One reason for doing this is to make an even-order moving average symmetric.

For example, we might take a moving average of order 4, and then apply another moving average of order 2 to the results. In the following table, this has been done for the first few years of the Australian quarterly beer production data.

```
aus_production = pd.read_csv("../data/aus_production.csv",
    parse_dates=["ds"])
beer = (
    aus_production[["ds", "Beer"]]
    .loc[lambda x: x["ds"] >= "1992"]
    .reset_index(drop=True)
)
beer["4-MA"] = beer["Beer"].rolling(4, center=True).mean().shift(-1)
beer["2x4-MA"] = beer["4-MA"].rolling(2, center=True).mean()
```

Table 3.1: A moving average of order 4 applied to the quarterly beer data, followed by a moving average of order 2.

	ds	Beer	4-MA	2x4-MA
0	1992-01-01	443		
1	1992-04-01	410	451.25	
2	1992-07-01	420	448.75	450.00
3	1992-10-01	532	451.50	450.12
4	1993-01-01	433	449.00	450.25
5	1993-04-01	421	444.00	446.50
	...	...	...	...
68	2009-01-01	415	430.00	428.88
69	2009-04-01	398	430.00	430.00
70	2009-07-01	419	429.75	429.88
71	2009-10-01	488	423.75	426.75
72	2010-01-01	414		
73	2010-04-01	374		

The notation "2×4-MA" in the last column means a 4-MA followed by a 2-MA. The values in the last column are obtained by taking a moving average of order 2 of the values in the previous column. For example, the first two values in the 4-MA column are $451.25 = (443 + 410 + 420 + 532)/4$ and $448.75 = (410 + 420 + 532 + 433)/4$ The first value in the 2x4-MA column is the average of these two: $450 = (451.25 + 448.75)/2$.

When a 2-MA follows a moving average of an even order (such as 4), it is called a "centred moving average of order 4". This is because the results are now symmetric. To see that this is the case, we can write the 2×4-MA as follows:

$$\begin{aligned}\hat{T}_t &= \frac{1}{2}\left[\frac{1}{4}(y_{t-2} + y_{t-1} + y_t + y_{t+1}) + \frac{1}{4}(y_{t-1} + y_t + y_{t+1} + y_{t+2})\right] \\ &= \frac{1}{8}y_{t-2} + \frac{1}{4}y_{t-1} + \frac{1}{4}y_t + \frac{1}{4}y_{t+1} + \frac{1}{8}y_{t+2}.\end{aligned}$$

It is now a weighted average of observations that is symmetric.

Other combinations of moving averages are also possible. For example, a 3×3-MA is often used, and consists of a moving average of order 3 followed by another moving average of order 3. In general, an even order MA should be followed by an even order MA to make it symmetric. Similarly, an odd order MA should be followed by an odd order MA.

Estimating the trend-cycle with seasonal data

The most common use of centred moving averages is for estimating the trend-cycle from seasonal data. Consider the 2×4-MA:

$$\hat{T}_t = \frac{1}{8}y_{t-2} + \frac{1}{4}y_{t-1} + \frac{1}{4}y_t + \frac{1}{4}y_{t+1} + \frac{1}{8}y_{t+2}.$$

When applied to quarterly data, each quarter of the year is given equal weight as the first and last terms apply to the same quarter in consecutive years. Consequently, the seasonal variation will be averaged out and the resulting values of $\hat{T}_t$ will have little or no seasonal variation remaining. A similar effect would be obtained using a 2×8-MA or a 2×12-MA to quarterly data.

In general, a $2 \times m$-MA is equivalent to a weighted moving average of order $m + 1$ where all observations take the weight $1/m$, except for the first and last terms which take weights $1/(2m)$. So, if the seasonal period is even and of order m, we use a $2 \times m$-MA to estimate the trend-cycle. If the seasonal period is odd and of order m, we use a m-MA to estimate the trend-cycle. For example, a 2×12-MA can be used to estimate the trend-cycle of monthly data with annual seasonality and a 7-MA can be used to estimate the trend-cycle of daily data with a weekly seasonality.

Other choices for the order of the MA will usually result in trend-cycle estimates being contaminated by the seasonality in the data.

Example: Employment in the US retail sector

```
df = us_retail_employment.copy()
df["12-MA"] = df["y"].rolling(window=12, center=True).mean()
df["2x12-MA"] = df["12-MA"].rolling(window=2, center=True).mean()

fig, ax = plt.subplots()
sns.lineplot(data=df, x="ds", y="y",
    color="grey", label="Data")
sns.lineplot(data=df, x="ds", y="2x12-MA",
    color="#D55E00", label="2x12-MA")
ax.set(
    title="Total employment in US retail",
    xlabel="Month [1M]",
    ylabel="Persons (thousands)",
)
plt.show()
```

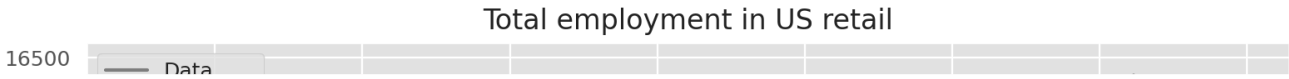


Figure 3.10: A 2x12-MA applied to the US retail employment series.

Figure 3.10 shows a 2×12-MA applied to the total number of persons employed in the US retail sector. Notice that the smooth line shows no seasonality; it is almost the same as the trend-cycle shown in Figure 3.4, which was estimated using a much more sophisticated method than a moving average. Any other choice for the order of the moving average (except for 24, 36, etc.) would have resulted in a smooth line that showed some seasonal fluctuations.

Weighted moving averages

Combinations of moving averages result in weighted moving averages. For example, the 2×4-MA discussed above is equivalent to a weighted 5-MA with weights given by $\left[\frac{1}{8}, \frac{1}{4}, \frac{1}{4}, \frac{1}{4}, \frac{1}{8}\right]$. In general, a weighted m-MA can be written as

$$\hat{T}_t = \sum_{j=-k}^{k} a_j y_{t+j},$$

where $k = (m-1)/2$, and the weights are given by $[a_{-k}, \ldots, a_k]$. It is important that the weights all sum to one and that they are symmetric so that $a_j = a_{-j}$. The simple m-MA is a special case where all of the weights are equal to $1/m$.

A major advantage of weighted moving averages is that they yield a smoother estimate of the trend-cycle. Instead of observations entering and leaving the calculation at full weight, their weights slowly increase and then slowly decrease, resulting in a smoother curve.

3.4 Classical decomposition

The classical decomposition method originated in the 1920s. It is a relatively simple procedure, and forms the starting point for most other methods of time series decomposition. There are two forms of classical decomposition: an additive decomposition and a multiplicative decomposition. These are described below for a time series with seasonal period m (e.g., $m = 4$ for quarterly data, $m = 12$ for monthly data, $m = 7$ for daily data with a weekly pattern).

In classical decomposition, we assume that the seasonal component is constant from year to year. For multiplicative seasonality, the m values that form the seasonal component are sometimes called the "seasonal indices".

Additive decomposition

Step 1

If m is an even number, compute the trend-cycle component $\hat{T}_t$ using a $2 \times m$-MA. If m is an odd number, compute the trend-cycle component $\hat{T}_t$ using an m-MA.

Step 2

Calculate the detrended series: $y_t - \hat{T}_t$.

Step 3

To estimate the seasonal component for each season, simply average the detrended values for that season. For example, with monthly data, the seasonal component for March is the average of all the detrended March values in the data. These seasonal component values are then adjusted to ensure that they add to zero. The seasonal component is obtained by stringing together these monthly values, and then replicating the sequence for each year of data. This gives $\hat{S}_t$.

Step 4

The remainder component is calculated by subtracting the estimated seasonal and trend-cycle components: $\hat{R}_t = y_t - \hat{T}_t - \hat{S}_t$.

Figure 3.11 shows a classical decomposition of the total retail employment series across the US.

Figure 3.11: A classical additive decomposition of US retail employment.

Multiplicative decomposition

A classical multiplicative decomposition is similar, except that the subtractions are replaced by divisions.

Step 1

If m is an even number, compute the trend-cycle component $\hat{T}_t$ using a $2 \times m$-MA. If m is an odd number, compute the trend-cycle component $\hat{T}_t$ using an m-MA.

Step 2

Calculate the detrended series: $y_t / \hat{T}_t$.

Step 3

To estimate the seasonal component for each season, simply average the detrended values for that season. For example, with monthly data, the seasonal index for March is the average of all the detrended March values in the data. These seasonal indexes are then adjusted to ensure that they add to m. The seasonal component is obtained by stringing together these monthly indexes, and then replicating the sequence for each year of data. This gives $\hat{S}_t$.

Step 4

The remainder component is calculated by dividing out the estimated seasonal and trend-cycle components: $\hat{R}_t = y_t/(\hat{T}_t\hat{S}_t)$.

Comments on classical decomposition

While classical decomposition is still widely used, it is not recommended, as there are now several much better methods. Some of the problems with classical decomposition are summarised below.

- The estimate of the trend-cycle is unavailable for the first few and last few observations. For example, if $m = 12$, there is no trend-cycle estimate for the first six or the last six observations. Consequently, there is also no estimate of the remainder component for the same time periods.
- The trend-cycle estimate tends to over-smooth rapid rises and falls in the data.
- Classical decomposition methods assume that the seasonal component repeats from year to year. For many series, this is a reasonable assumption, but for some longer series it is not. For example, electricity demand patterns have changed over time as air conditioning has become more widespread. In many locations, the seasonal usage pattern from several decades ago had its maximum demand in winter (due to heating), while the current seasonal pattern has its maximum demand in summer (due to air conditioning). Classical decomposition methods are unable to capture these seasonal changes over time.
- Occasionally, the values of the time series in a small number of periods may be particularly unusual. For example, the monthly air passenger traffic may be affected by an industrial dispute, making the traffic during the dispute different from usual. The classical method is not robust to these kinds of unusual values.

3.5 Methods used by official statistics agencies

Official statistics agencies (such as the US Census Bureau and the Australian Bureau of Statistics) are responsible for a large number of official economic and social time series. These agencies have developed their own decomposition procedures which are used for seasonal adjustment. Most of them use variants of the X-11 method, or the SEATS method, or a combination of the two. These methods are designed specifically to work with quarterly and monthly data, which are the most common series handled by official statistics agencies. They will not handle seasonality of other kinds, such as daily data, or hourly data, or weekly data.

X-11 method

The X-11 method originated in the US Census Bureau and was further developed by Statistics Canada. It is based on classical decomposition, but includes many extra steps and features in order to overcome the drawbacks of classical decomposition that were discussed in the previous

section. In particular, trend-cycle estimates are available for all observations including the end points, and the seasonal component is allowed to vary slowly over time. X-11 also handles trading day variation, holiday effects and the effects of known predictors. There are methods for both additive and multiplicative decomposition. The process is entirely automatic and tends to be highly robust to outliers and level shifts in the time series. The details of the X-11 method are described in Dagum and Bianconcini (2016).

SEATS method

"SEATS" stands for "Seasonal Extraction in ARIMA Time Series" (ARIMA models are discussed in Chapter 9). This procedure was developed at the Bank of Spain, and is now widely used by government agencies around the world. The details are beyond the scope of this book. However, a complete discussion of the method is available in Dagum and Bianconcini (2016).

3.6 STL decomposition

STL is a versatile and robust method for decomposing time series. STL is an acronym for "Seasonal and Trend decomposition using Loess", while loess is a method for estimating nonlinear relationships. The STL method was developed by Cleveland et al. (1990), and later extended to handle multiple seasonal patterns by Bandara, Hyndman, and Bergmeir (2022).

STL has several advantages over classical decomposition, and the SEATS and X-11 methods:

- Unlike SEATS and X-11, STL will handle any type of seasonality, not only monthly and quarterly data.
- The seasonal component is allowed to change over time, and the rate of change can be controlled by the user.
- The smoothness of the trend-cycle can also be controlled by the user.
- It can be robust to outliers (i.e., the user can specify a robust decomposition), so that occasional unusual observations will not affect the estimates of the trend-cycle and seasonal components. They will, however, affect the remainder component.

On the other hand, STL has some disadvantages. In particular, it does not handle trading day or calendar variation automatically, and it only provides facilities for additive decompositions.

A multiplicative decomposition can be obtained by first taking logs of the data, then back-transforming the components. Decompositions that are between additive and multiplicative can be obtained using a Box-Cox transformation of the data with $0 < \lambda < 1$. A value of $\lambda = 0$ gives a multiplicative decomposition while $\lambda = 1$ gives an additive decomposition.

The best way to begin learning how to use STL is to see some examples and experiment with the settings. Figure 3.5 showed an example of an STL decomposition applied to the total US

retail employment series. Figure 3.12 shows an alternative STL decomposition where the trend-cycle is more flexible, the seasonal pattern is fixed, and the robust option has been used.

```
stl = STL(us_retail_employment["y"], period=12,
    seasonal=13, trend=21, robust=True)
res_stl = stl.fit()
dcmp = pd.DataFrame({
    "ds": us_retail_employment["ds"],
    "data": us_retail_employment["y"],
    "trend": res_stl.trend,
    "seasonal": res_stl.seasonal,
    "remainder": res_stl.resid,
}).reset_index(drop=True)

fig, axes = plt.subplots(4, 1, sharex=True, figsize=(8, 6))
sns.lineplot(data=dcmp, x="ds", y="data", ax=axes[0])
sns.lineplot(data=dcmp, x="ds", y="trend", ax=axes[1])
sns.lineplot(data=dcmp, x="ds", y="seasonal", ax=axes[2])
sns.lineplot(data=dcmp, x="ds", y="remainder", ax=axes[3])
axes[0].set_title("Employed = trend + seasonal + random",
    size="medium", loc="left")
axes[0].set(ylabel="Employed")
axes[3].set(xlabel="")
fig.suptitle("STL decomposition")
plt.show()
```

Figure 3.12: Total US retail employment (top) and its three additive components obtained from a robust STL decomposition with flexible trend-cycle and fixed seasonality.

The two main parameters to be chosen when using STL are the `season` and the `trend`. The `season` is the length of the seasonal smoother, while the `trend` is the length of the trend smoother. These parameters control how rapidly the trend-cycle and seasonal components can change. Smaller values allow for more rapid changes. Both trend and seasonal windows should be odd numbers.

By default, the `STL()` function in `statsmodels` provides a convenient automated STL decomposition. This usually gives a good balance between overfitting the seasonality and allowing it to slowly change over time. But, as with any automated procedure, the default settings will need adjusting for some time series. In this case the default trend window setting produces a trend-cycle component that is too rigid. As a result, signal from the 2008 global financial crisis has leaked into the remainder component, as can be seen in the bottom panel of Figure 3.5. Selecting a shorter trend window as in Figure 3.12 improves this.

3.7 Exercises

1. Consider the GDP information in `global_economy`. Plot the GDP per capita for each country over time. Which country has the highest GDP per capita? How has this changed over time?

2. For each of the following series, make a graph of the data. If transforming seems appropriate, do so and describe the effect.

   - United States GDP from `global_economy`.
   - Slaughter of Victorian "Bulls, bullocks and steers" in `aus_livestock`.
   - Victorian Electricity Demand from `vic_elec`.
   - Gas production from `aus_production`.

3. Why is a Box-Cox transformation unhelpful for the `canadian_gas` data?

4. What Box-Cox transformation would you select for your retail data (from Exercise 7 in Section 2.10)?

5. For the following series, find an appropriate Box-Cox transformation in order to stabilise the variance. Tobacco from `aus_production`, Economy class passengers between Melbourne and Sydney from `ansett`, and Pedestrian counts at Southern Cross Station from `pedestrian`.

6. Show that a 3×5 MA is equivalent to a 7-term weighted moving average with weights of 0.067, 0.133, 0.200, 0.200, 0.200, 0.133, and 0.067.

7. Consider the last five years of the Gas data from `aus_production`.

```
gas = aus_production.loc[lambda x: x["unique_id"] == "Gas"].tail(5 * 4)
```

   a. Plot the time series. Can you identify seasonal fluctuations and/or a trend-cycle?
   b. Use `seasonal_decompose` with `model='multiplicative'` to calculate the trend-cycle and seasonal indices.
   c. Do the results support the graphical interpretation from part a?
   d. Compute and plot the seasonally adjusted data.
   e. Change one observation to be an outlier (e.g., add 300 to one observation), and recompute the seasonally adjusted data. What is the effect of the outlier?
   f. Does it make any difference if the outlier is near the end rather than in the middle of the time series?

8. Recall your retail time series data (from Exercise 7 in Section 2.10). Decompose the series using the MSTL method. Does it reveal any outliers, or unusual features that you had not noticed previously?

9. Figures 3.13 and 3.14 show the result of decomposing the number of persons in the civilian labour force in Australia each month from February 1978 to August 1995.

   a. Write about 3–5 sentences describing the results of the decomposition. Pay particular attention to the scales of the graphs in making your interpretation.
   b. Is the recession of 1991/1992 visible in the estimated components?

Figure 3.13: Decomposition of the number of persons in the civilian labour force in Australia each month from February 1978 to August 1995.

Figure 3.14: Seasonal component from the decomposition shown in the previous figure.

10. This exercise uses the `canadian_gas` data (monthly Canadian gas production in billions of cubic metres, January 1960 – February 2005).
    a. Plot the data using to look at the effect of the changing seasonality over time.[1]
    b. Do an STL decomposition of the data using `STL()`.
    c. Como o formato sazonal muda ao longo do tempo? [Dica: tente plotar o componente sazonal.]
    d. Você consegue produzir uma série temporal ajustada sazonalmente plausível?

3.8 Leitura complementar

- Cleveland et al. (1990) introduziram o STL e ainda fornecem a melhor descrição do algoritmo. Bandara, Hyndman e Bergmeir (2022) introduziram o MSTL, que é uma extensão do STL que permite múltiplos padrões sazonais.

3.9 Módulos e classes utilizados

Previsão da Core

- `boxcox()` Função - Para aplicar transformações de Box-Cox
- `boxcox_lambda()` Função - Para encontrar os parâmetros ideais da transformação de Box-Cox

Previsão de utilidades

- `plot_series()` Utilitário - Para criar visualizações de séries temporais

---

1. A evolução do padrão sazonal possivelmente se deve a mudanças na regulamentação dos preços do gás — agradecemos a Lewis Kirvan por apontar isso. ↩

Publicado pela OTexts usando Quarto . © 2026